

UNITAT 6 · SETMANES 8–9

Senyals i memòria compartida

📅 Setmanes 8–9 · dates per confirmar </> Laboratori TD5 (resolt + repte IA) ~ 3 animacions

MANUAL M6 · TEORIA DE LA UNITAT

Senyals i memòria compartida

Lectura bàsica de la unitat 6 ($\approx 1,5$ h). Les sortides són reals: programes del TD5 executats a la màquina del curs.

CONTINGUT

- | | | | |
|----|---|----|--|
| §1 | Senyals: enviament, pendents i lliurament | §3 | <code>mmap</code> : mapar fitxers a la memòria |
| §2 | Gestors i sincronització sense curses | §4 | La memòria compartida i el seu preu |

§1 Senyals: enviament, pendents i lliurament

Un **senyal** és una notificació asíncrona a un procés. L'envia el nucli (SIGSEGV per un accés no vàlid, SIGPIPE per escriure en una canonada sense lector, SIGCHLD quan un fill acaba), el terminal (Ctrl-C → SIGINT, Ctrl-\ → SIGQUIT, Ctrl-Z → SIGTSTP) o un altre procés amb `kill(pid, sig)`. Per a cada senyal, el procés té una **acció**: la per defecte (acabar, acabar amb core, aturar, continuar o ignorar), ignorar-lo (`SIG_IGN`) o executar una funció **gestora**. `SIGKILL` i `SIGSTOP` són els únics que no es poden capturar, ignorar ni bloquejar.

`kill` només marca el senyal com a **pendent**. El nucli el **lliura** quan el procés torna al mode usuari —quan acaba una crida al sistema o quan obté la CPU després d'un canvi de context—, tret que estigui **bloquejat** per la màscara del procés: llavors espera. Per als senyals estàndard, «pendent» és un bit: si n'arriben tres d'ídèntics mentre està bloquejat, en desbloquejar-lo només se'n lliura un. Els senyals de temps real (`SIGRTMIN ...`) sí que fan cua i poden portar un valor.

Practica-ho → A6.1 lliurament de senyals

§2 Gestors i sincronització sense curses

Els gestors s'instal·len amb `sigaction` (`signal()` té semàntiques diferents segons el sistema). Mentre el gestor s'executa, el mateix senyal queda bloquejat (i també els de `sa_mask`); amb `SA_RESTART`, una crida al sistema interrompuda es reprèn sola; si no, falla amb `EINTR` (unitat 2). Un gestor interromp el programa en qualsevol punt, així que només pot fer servir funcions *segures davant senyals asíncrons* (`write` sí; `printf` i `malloc` no) i comunicar-se amb el programa a través de variables `volatile sig_atomic_t`.

Esperar un senyal amb `pause()` té una **cursa**: si el senyal arriba entre l'última comprovació i la crida a `pause`, el gestor s'executa abans d'hora i `pause` espera per sempre. La solució és **bloquejar** el senyal amb `sigprocmask`, comprovar la condició i esperar amb `sigsuspend(mask)`, que substitueix la màscara i dorm **en una sola operació atòmica**: el senyal que havia arribat «aviat» ha quedat pendent i es lliura dins de `sigsuspend`.

C tds/td5/mysignal.c

```
/*
 * TD5 - exercise 1: catch SIGUSR1 with sigaction(2).
 * ./mysignal [n]      handles SIGUSR1 n times (default 3), then restores the default
 * action
 * From another terminal: kill -USR1 <pid>, kill -STOP <pid>, kill -CONT <pid>, kill
 * -KILL <pid>
 * After the n-th SIGUSR1 the default action is back: the next SIGUSR1 terminates the
 * process.
 */
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static volatile sig_atomic_t received = 0;

static void on_usr1(int sig)
{
    /* Only async-signal-safe functions here: no printf (it may be interrupted mid-
    buffer). */
    static const char msg[] = "handler: received signal ";
    char digits[4] = { (char)('0' + sig / 10), (char)('0' + sig % 10), '\n', 0 };
    if (write_all(STDERR_FILENO, msg, sizeof msg - 1) == -1 || write_all(STDERR_FILENO,
    digits, 3) == -1)
        return;
    received++;
}

int main(int argc, char *argv[])
{
    int max = argc > 1 ? atoi(argv[1]) : 3;
    struct sigaction sa = { 0 }, old;
    sigset_t block, wait_mask;

    printf("pid %d: waiting for SIGUSR1 (%d times)\n", getpid(), max);
    fflush(stdout);
```

```

sa.sa_handler = on_usr1;
sigemptyset(&sa.sa_mask);          /* nothing extra blocked while the handler runs
*/
sa.sa_flags = 0;
exit_if(sigaction(SIGUSR1, &sa, &old) == -1, "sigaction");

/* Block SIGUSR1 outside sigsuspend: a signal arriving between the test of
`received`
and the wait stays pending instead of being lost (the pause() race). */
sigemptyset(&block);
sigaddset(&block, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &block, &wait_mask) == -1, "sigprocmask");
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unblock SIGUSR1 and sleep */

exit_if(sigaction(SIGUSR1, &old, NULL) == -1, "sigaction restore");
printf("default action restored: the next SIGUSR1 terminates me\n");
fflush(stdout);
exit_if(sigprocmask(SIG_UNBLOCK, &block, NULL) == -1, "sigprocmask");
for (;;)
    pause();
}

```

Sortida real

```

$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep 0.2; kill -STOP $P; kill -USR1
$P; sleep 0.2; kill -CONT $P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status: $?
(128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1      ./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)

```

Altres eines: `alarm(s)` programa un `SIGALRM`; el `sleep` de glibc fa servir `clock_nanosleep` (no `alarm`); `signalfd` converteix els senyals en dades que es llegeixen d'un descriptor, i `pidfd_send_signal` evita enviar un senyal a un PID que ja s'ha reutilitzat.

Practica-ho → TD5 · 1 i 2

§3 mmap : mapar fitxers a la memòria

`mmap(addr, len, prot, flags, fd, offset)` reserva un rang d'adreces i el lliga a un fitxer (o a res, amb `MAP_ANONYMOUS`). No copia res: cada pàgina es porta en el primer accés amb una fallada de pàgina (unitat 4). Amb `MAP_SHARED` les escriptures van a la memòria cau de pàgines del fitxer, els altres processos que el mapen les veuen i arriben al disc (`msync` o en desfer el mapatge); amb `MAP_PRIVATE` la primera escriptura copia la pàgina i el fitxer no canvia. `len` s'arrodoneix cap amunt a pàgines; `offset` ha de ser múltiple de la mida de pàgina.

Accedir més enllà del final del fitxer dins de l'última pàgina retorna zeros que no es desen; tocar una pàgina sencera sense fitxer al darrere provoca **SIGBUS** (l'adreça és vàlida, però no hi ha dades per portar). Per fer créixer un fitxer mapat, crida primer `ftruncate`. Les proteccions `PROT_*` s'apliquen per pàgina; a x86, escriure implica poder llegir:

Sortida real

```
$ ./mmap_limits
file size 128 bytes, page size 4096 bytes, mapping of 8192 bytes
wrote bytes 0..4095; byte 4096 raised SIGBUS
file size after the writes: 128 bytes
```

Sortida real

```
$ ./map_write
PROT = PROT_WRITE
pointer: 0x7c7095214000
read:    *p = 0
write:   *p = 42
```

Sortida real

```
$ ./map_exec
PROT = PROT_EXEC
pointer: 0x75b3a49d8000
bash: line 1: 357495 Segmentation fault      (core dumped) ./map_exec
[exit status 139]
```

Practica-ho → A6.3 els límits de mmap · TD5 · 3 i 4

§4 La memòria compartida i el seu preu

Compartir memòria vol dir fer que pàgines virtuals de dos processos apuntin al mateix marc físic. Calen un **punt de trobada** i una interfície: `mmap` d'un fitxer comú amb `MAP_SHARED`; `shm_open` de POSIX + `mmap` (un nom a `/dev/shm`, recomanat avui); o System V: `ftok(file, id)` genera una

clau, `shmget` crea o troba el segment, `shmat` l'enganxa, `shmdt` el desenganxa i `shmctl(IPC_RMID)` el destrueix. El segment sobreviu als processos fins que algú el destrueix:

Sortida real

```
$ ./shm_counter; ./shm_counter; ./shm_counter; ./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

Dues regles. Primera: cada procés veu la zona en una **adreça diferent**, així que no s'hi guarden punters (sinó desplaçaments). Segona: la memòria compartida és la comunicació més ràpida perquè no passa pel nucli, però precisament per això **res no la sincronitza**. Si diversos processos fan «llegir, sumar 1, escriure» sobre el mateix comptador, es perden actualitzacions; amb una operació atòmica del processador (`__atomic_add_fetch`, `lock xadd` a x86), no:

Sortida real

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

Sortida real

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566 357567 357572 357568 357571 357569
357565 357574 357573 357570 357576 357578 357575 357577 357579
```

Practica-ho → A6.2 memòria compartida · TD5 · 5